MultiLabelMarginLoss#

`https://pytorch.org/docs/stable/generated/torch.nn.modules.loss.MultiLabelMa`

Description:

Creates a criterion that optimizes a multi-class multi-classification hinge loss (margin-based loss) between input xxx (a 2D mini-batch Tensor) and output yyy (which is a 2D Tensor of target class indices). For each sample in the mini-batch: where x∈{0, ⋯, x.size(0)−1}x \in \left\{0, \; \cdots , \; \text{x.size}(0) - 1\right\}x∈{0,⋯,x.size(0)−1}, y∈{0, ⋯, y.size(0)−1}y \in \left\{0, \; \cdots , \; \text{y.size}(0) - 1\right\}y∈{0,⋯,y.size(0)−1}, 0≤y[j]≤x.size(0)−10 \leq y[j] \leq \text{x.size}(0)-10≤y[j]≤x.size(0)−1, and i≠y[j]i \neq y[j]i=y[j] for all iii and jjj. yyy and xxx must have the same size. The criterion only considers a contiguous block of non-negative targets that starts at the front. This allows for different samples to have variable amounts of target classes.

Function Signature

```
class
torch.nn.modules.loss.MultiLabelMarginLoss(size_average=None
, reduce=None, reduction='mean')[source]#

Parameters

Shape:

forward(input, target)[source]#

Return type
```

Returns:

Tensor

Code Examples

```
>>> loss = nn.MultiLabelMarginLoss()
>>> x = torch.FloatTensor([[0.1, 0.2, 0.4, 0.8]])
>>> # for target y, only consider labels 3 and 0, not after
label -1
>>> y = torch.LongTensor([[3, 0, -1, 1]])
>>> # 0.25 * ((1-(0.1-0.2)) + (1-(0.1-0.4)) + (1-(0.8-0.2)) +
(1-(0.8-0.4)))
>>> loss(x, y)
tensor(0.85...)
```

Detailed Documentation

Documentation

Creates a criterion that optimizes a multi-class multi-classification hinge loss (margin-based loss) between input xxx (a 2D mini-batch Tensor) and output yyy (which is a 2D Tensor of target class indices). For each sample in the mini-batch:

where x∈{0, ⋯, x.size(0)−1}x \in \left\{0, \; \cdots , \; \text{x.size}(0) - 1\right\}x∈{0,⋯,x.size(0)−1}, y∈{0, ⋯, y.size(0)−1}y \in \left\{0, \; \cdots , \; \text{y.size}(0) - 1\right\}y∈{0,⋯,y.size(0)−1}, 0≤y[j]≤x.size(0)−10 \leq y[j] \leq \text{x.size}(0)-10≤y[j]≤x.size(0)−1, and i≠y[j]i \neq y[j]i=y[j] for all iii and jjj.

yyy and xxx must have the same size.

The criterion only considers a contiguous block of non-negative targets that starts at the front.

This allows for different samples to have variable amounts of target classes.

size_average (bool, optional) – Deprecated (see reduction). By default, the losses are averaged over each loss element in the batch. Note that for some losses, there are multiple elements per sample. If the field size_average is set to False, the losses are instead summed for each minibatch. Ignored when reduce is False. Default: True

reduce (bool, optional) – Deprecated (see reduction). By default, the losses are averaged or summed over observations for each minibatch depending on size_average. When reduce is False, returns a loss per batch element instead and ignores size_average. Default: True reduction (str, optional) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the sum of the output will be divided by the number of elements in the output, 'sum': the output will be summed. Note: size_average and reduce are in the process of being deprecated, and in the meantime, specifying either of those two args will override reduction. Default: 'mean'

Input: (C)(C)(C) or (N,C)(N, C)(N,C) where N is the batch size and C is the number of classes.

Target: (C)(C)(C) or (N,C)(N, C)(N,C), label targets padded by -1 ensuring same shape as the input.

Output: scalar. If reduction is 'none', then (N)(N)(N).

Runs the forward pass.